

EXERCÍCIO PRÁTICO RESOLVIDO

Pré-processamento de dados em IoT

Limpeza, ruídos, valores ausentes, transformação e padronização

Disciplina: Mineração de dados e internet das coisas

Professor Msc Elionai Magalhães

Estudo de caso fictício: Sala Inteligente B-204

Importante: Todos os dados deste exercício são fictícios e foram criados exclusivamente para fins didáticos. Não representam medições reais de pessoas, salas ou equipamentos.

Objetivo: acompanhar, no Google Colab ou em instalação local do Python, um pipeline completo de diagnóstico, limpeza, tratamento de ruído, criação de atributos e ajuste de escala.

1. Enunciado do exercício

Uma instituição de ensino instalou sensores em uma sala inteligente para acompanhar conforto térmico e consumo de energia. A sala B-204 envia uma leitura a cada cinco minutos contendo temperatura, umidade, presença e potência média consumida. O objetivo da equipe de dados é preparar a base para análises posteriores de mineração de dados, identificando períodos de possível desconforto e indícios de consumo quando a sala está vazia.

Durante a coleta, ocorreram falhas controladas: uma temperatura ausente, uma leitura impossível, uma medição em Fahrenheit no meio de valores em Celsius, umidade ausente, um registro duplicado e duas linhas fora de ordem temporal. O estudante deverá diagnosticar e tratar esses problemas sem apagar automaticamente eventos que possam ser relevantes.

Objetivos da prática

- Carregar e inspecionar uma base CSV de sensores.
- Detectar duplicidades, ausências, valores fora de faixa e inconsistências de unidade.
- Converter timestamps, ordenar registros e padronizar a unidade de temperatura.
- Tratar valores ausentes por interpolação temporal e suavizar ruído com média móvel.
- Criar atributos derivados, como hora, turno, energia por intervalo e indicador de desperdício.
- Aplicar normalização e padronização sem vazamento de dados.
- Comparar a base antes e depois e interpretar os resultados.

Relação com a apresentação: O exercício reproduz a lógica do material de aula: diagnóstico → limpeza → ausentes → ruído → transformação → escala → validação.

2. Dados fictícios fornecidos

O arquivo `dados_sala_inteligente_brutos.csv` contém 25 linhas. Cada linha representa uma leitura de cinco minutos. Os campos são: `timestamp`, `sensorId`, `temp`, `unidade_temp`, `umid`, `presenca`, `consumo_kw` e `status`.

timestamp	sensorId	temp	unidade_temp	umid	presenca	consumo_kw	status
2026-09-01 08:00	sala_b204	23.1	C	62	1	0.8	ok
2026-09-01 08:05	sala_b204		C	63	1	0.83	falha
2026-09-01 08:10	sala_b204	240	C	64	1	0.85	alerta
2026-09-01 08:15	sala_b204	23.5	C	64	1	0.82	ok
2026-09-01 08:20	sala_b204	75.2	F	65	1	0.84	ok
2026-09-01 08:25	sala_b204	23.9	C		1	0.86	falha

2026-09-01 08:30	sala_b204	24.2	C	66	1	0.9	ok
2026-09-01 08:30	sala_b204	24.2	C	66	1	0.9	ok
2026-09-01 08:35	sala_b204	24.8	C	67	1	0.95	ok
2026-09-01 08:40	sala_b204	25.2	C	68	1	1	ok
2026-09-01 08:45	sala_b204	25.6	C	68	1	1.05	ok
2026-09-01 08:50	sala_b204	25.9	C	69	1	1.1	ok

Arquivo de apoio: O arquivo completo em CSV acompanha este documento. A tabela acima mostra apenas as primeiras linhas para facilitar a leitura.

3. Passo 1 — carregar e inspecionar a base

Código no Google Colab / Python

```
import pandas as pd

df = pd.read_csv('dados_sala_inteligente_brutos.csv')
print(df.head())
print(df.info())
print(df.isna().sum())
print('Duplicados:', df.duplicated().sum())
```

Resultado esperado do diagnóstico inicial

- A base possui 25 registros e 8 colunas.
- Existe 1 valor ausente em temperatura e 1 valor ausente em umidade.
- Existe 1 registro exatamente duplicado.
- Há 1 leitura de temperatura registrada em Fahrenheit, enquanto as demais estão em Celsius.
- Há 1 temperatura de 240 °C, incompatível com uma sala de aula e que deve ser investigada.
- Há pelo menos uma inversão na ordem temporal: 09:20 aparece antes de 09:15 no arquivo bruto.

Ponto didático: O número 75,2 não deve ser classificado como outlier apenas pelo valor numérico: a coluna `unidade_temp` informa que ele está em Fahrenheit. O contexto vem antes da exclusão.

4. Passo 2 — converter timestamp e ordenar

```
df['timestamp'] = pd.to_datetime(df['timestamp'])
df = df.sort_values('timestamp').reset_index(drop=True)

print(df[['timestamp', 'temp', 'unidade_temp']].head(8))
```

Ao converter timestamp para um tipo temporal, o pandas passa a entender data e hora. A ordenação é necessária antes de interpolar e calcular janelas móveis; caso contrário, médias e tendências podem ser calculadas sobre uma sequência incorreta.

5. Passo 3 — remover duplicidades

```
print('Antes:', len(df))
print('Duplicados:', df.duplicated().sum())
df = df.drop_duplicates().reset_index(drop=True)
print('Depois:', len(df))
```

Resultado: a base passa de 25 para 24 registros. Foi removida somente a repetição exata das 08:30. Em dados reais, a chave de comparação pode envolver timestamp, sensorId e identificador da mensagem.

6. Passo 4 — padronizar a unidade de temperatura

A leitura das 08:20 foi registrada como 75,2 °F. Antes de comparar com os demais valores, devemos convertê-la para Celsius.

```
mask_f = df['unidade_temp'].eq('F')
df.loc[mask_f, 'temp'] = (df.loc[mask_f, 'temp'] - 32) * 5/9
df.loc[mask_f, 'unidade_temp'] = 'C'
```

Cálculo manual

$C = (F - 32) \times 5/9 = (75,2 - 32) \times 5/9 = 24,0 \text{ °C}$. Assim, a leitura é perfeitamente plausível; o problema era apenas a unidade diferente.

7. Passo 5 — transformar valores impossíveis em ausentes

```
df.loc[(df['temp'] < 0) | (df['temp'] > 50), 'temp'] = pd.NA
print(df[['timestamp', 'temp']].head(6))
```

Para este estudo fictício, adotamos como regra operacional temperatura plausível entre 0 °C e 50 °C. O valor 240 °C é marcado como ausente, em vez de simplesmente substituído por um número arbitrário. Em outro domínio, os limites poderiam ser diferentes.

8. Passo 6 — tratar valores ausentes por interpolação

```
df['temp'] = df['temp'].interpolate(method='linear')
df['umid'] = df['umid'].interpolate(method='linear')
```

Após marcar 240 °C como inválido, temos duas lacunas consecutivas em temperatura: 08:05 e 08:10. A interpolação linear utiliza os valores válidos ao redor para estimar uma transição gradual entre 23,1 °C (08:00) e 23,5 °C (08:15).

timestamp	temp	umid	status
2026-09-01 08:00	23.1	62	ok
2026-09-01 08:05	23.233	63	falha
2026-09-01 08:10	23.367	64	alerta
2026-09-01 08:15	23.5	64	ok
2026-09-01 08:20	24	65	ok
2026-09-01 08:25	23.9	65.5	falha

Cuidado: A interpolação é uma escolha didática. Em situações reais, uma falha prolongada ou um evento abrupto pode exigir outro tratamento, inclusive manter a ausência explicitamente.

9. Passo 7 — suavizar pequenas oscilações com média móvel

Uma média móvel de três leituras resume o comportamento local sem substituir permanentemente a temperatura original. Mantemos as duas colunas: temp (tratada) e temp_mm3 (suavizada).

```
df['temp_mm3'] = df['temp'].rolling(3, min_periods=1).mean()
print(df[['timestamp', 'temp', 'temp_mm3']].head(10))
```

timestamp	temp	temp_mm3
2026-09-01 08:00	23.1	23.1
2026-09-01 08:05	23.233	23.167
2026-09-01 08:10	23.367	23.233
2026-09-01 08:15	23.5	23.367
2026-09-01 08:20	24	23.622
2026-09-01 08:25	23.9	23.8
2026-09-01 08:30	24.2	24.033
2026-09-01 08:35	24.8	24.3
2026-09-01 08:40	25.2	24.733
2026-09-01 08:45	25.6	25.2

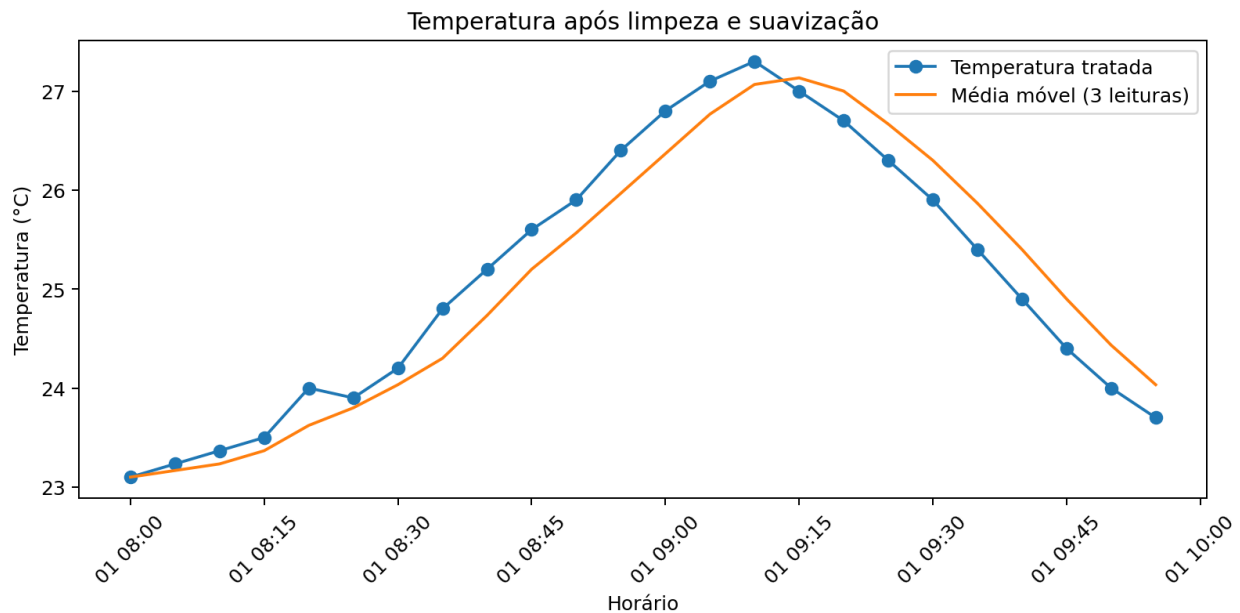


Figura 1 — Temperatura tratada e média móvel de três leituras.

10. Comparação visual: antes e depois

No gráfico bruto, os valores 240 e 75,2 aparecem como grandes picos porque ainda não consideramos a unidade nem a validade da medição. Após padronizar unidades, marcar o valor impossível, interpolar lacunas e calcular a média móvel, a série fica coerente com o cenário.

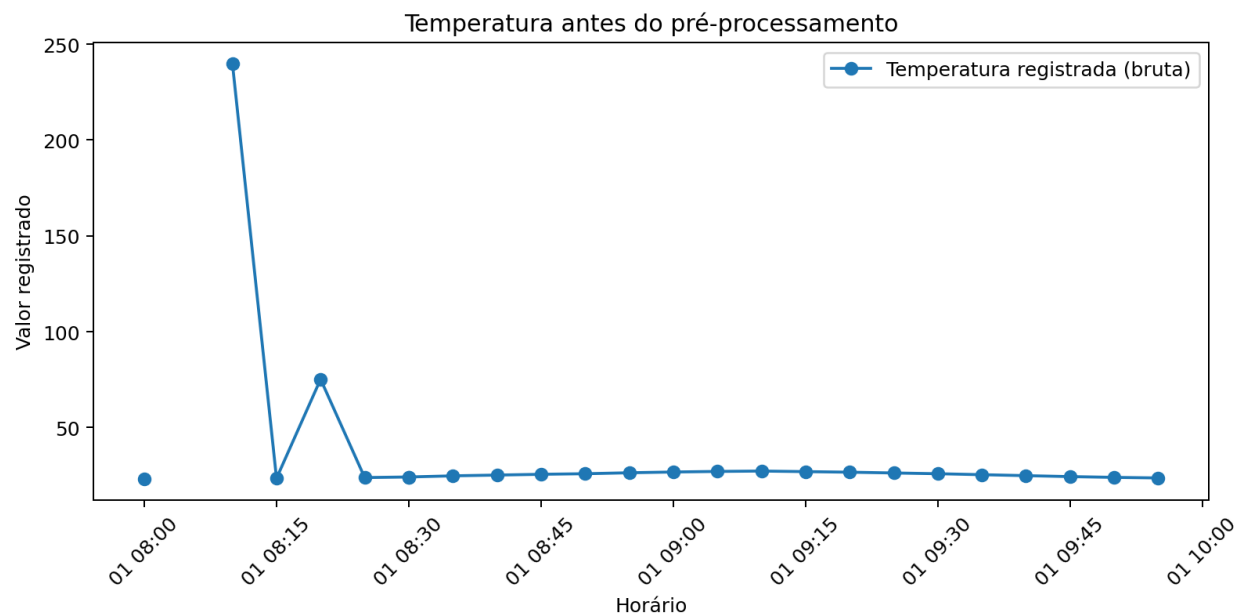


Figura 2 — Dados brutos: unidade misturada e valor impossível distorcem a escala.

11. Passo 8 — criar atributos úteis

```
df['hora'] = df['timestamp'].dt.hour
df['turno'] = 'manha' # todas as leituras deste exemplo são pela manhã
df['energia_kwh_intervalo'] = df['consumo_kw'] * (5/60)
df['desperdicio'] = ((df['presenca'] == 0) & (df['consumo_kw'] >= 0.80)).astype(int)
df['temp_variacao'] = df['temp'].diff().fillna(0)
```

Essas transformações criam informação que não estava explicitamente no arquivo. `energia_kwh_intervalo` estima a energia do intervalo de cinco minutos a partir da potência média. `desperdicio` é uma regra simples: sala vazia e consumo igual ou superior a 0,80 kW.

Resultado do indicador de possível desperdício

timestamp	presenca	consumo_kw	energia_kwh_intervalo	desperdicio
2026-09-01 09:15	0	0.95	0.079	1
2026-09-01 09:20	0	0.9	0.075	1
2026-09-01 09:25	0	0.85	0.071	1
2026-09-01 09:30	0	0.8	0.067	1

Foram encontrados 4 intervalos consecutivos de cinco minutos com sala vazia e potência média $\geq 0,80$ kW. A energia estimada nesses quatro intervalos é aproximadamente 0.292 kWh. Isso não prova desperdício por si só, mas cria um indício para investigação.

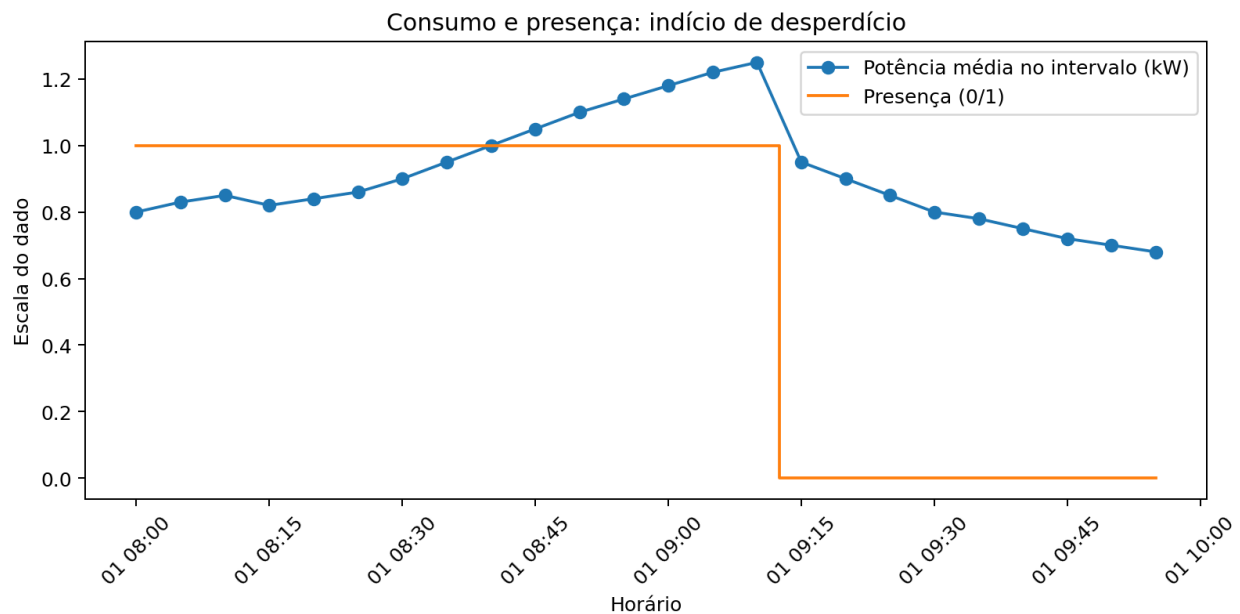


Figura 3 — A queda de presença ocorre antes de o consumo reduzir completamente.

12. Passo 9 — agregar em janelas de 15 minutos

```
resumo15 = (df.set_index('timestamp')
            .resample('15min'))
```



```

        .agg({'temp': 'mean',
              'umid': 'mean',
              'consumo_kw': 'mean',
              'presenca': 'max',
              'desperdicio': 'sum'})

print(resumo15)

```

timestamp	temp	umid	consumo_kw	presenca	desperdicio
2026-09-01 08:00	23.233	63	0.827	1	0
2026-09-01 08:15	23.8	64.833	0.84	1	0
2026-09-01 08:30	24.733	67	0.95	1	0
2026-09-01 08:45	25.967	69	1.097	1	0
2026-09-01 09:00	27.067	71	1.217	1	0
2026-09-01 09:15	26.667	71	0.9	0	3
2026-09-01 09:30	25.4	68	0.777	0	1
2026-09-01 09:45	24.033	65	0.7	0	0

Escolha da janela: A média é adequada para resumir tendência, mas pode esconder picos. Em outros casos, máximo, mínimo ou contagem de eventos pode ser mais informativo.

13. Passo 10 — separar treino e teste no tempo

Como os dados são uma série temporal, a separação preserva a ordem: os 75% registros mais antigos formam o treino e os 25% mais recentes formam o teste. Não embaralhamos aleatoriamente.

```
split = int(len(df) * 0.75)
treino = df.iloc[:split].copy()
teste = df.iloc[split:].copy()

print('Treino até:', treino['timestamp'].max())
print('Teste a partir de:', teste['timestamp'].min())
```

Neste conjunto, o treino contém 18 registros, encerrando em 09:25; o teste contém 6 registros e começa em 09:30.

14. Passo 11 — padronizar sem vazamento de dados

```
from sklearn.preprocessing import StandardScaler

colunas = ['temp_mm3', 'umid', 'consumo_kw']
scaler = StandardScaler()
scaler.fit(treino[colunas])

treino[colunas] = scaler.transform(treino[colunas])
teste[colunas] = scaler.transform(teste[colunas])
```

O ponto essencial é executar fit somente no treino. Se média e desvio padrão fossem calculados com toda a base, o período de teste influenciaria o preparo dos dados e ocorreria vazamento de informação.

variavel	media_treino	desvio_treino	min_treino	max_treino
temp_mm3	25.06	1.48	23.1	27.133
umid	67.639	3.108	62	72
consumo_kw	0.972	0.145	0.8	1.25

15. Normalização Min-Max: comparação opcional

```
from sklearn.preprocessing import MinMaxScaler

minmax = MinMaxScaler()
minmax.fit(treino[colunas])
treino_mm = minmax.transform(treino[colunas])
teste_mm = minmax.transform(teste[colunas])
```

A normalização coloca os valores do treino em um intervalo comum, normalmente de 0 a 1. Já a padronização indica distância em desvios padrão em relação à média. O exercício mostra ambos para comparação, mas um projeto deve escolher a transformação coerente com o algoritmo e com o problema.

Interpretação: O conjunto de teste pode gerar valores Min-Max fora do intervalo [0,1] quando contém valores abaixo do mínimo ou acima do máximo observados no treino. Isso não é erro: significa que o teste trouxe uma condição nova em relação ao treino.

16. Base preparada — amostra do resultado

timestamp	temp	umid	temp_mm3	presenca	consumo_kw	desperdicio
2026-09-01 08:00	23.1	62	23.1	1	0.8	0
2026-09-01 08:05	23.233	63	23.167	1	0.83	0
2026-09-01 08:10	23.367	64	23.233	1	0.85	0
2026-09-01 08:15	23.5	64	23.367	1	0.82	0
2026-09-01 08:20	24	65	23.622	1	0.84	0
2026-09-01 08:25	23.9	65.5	23.8	1	0.86	0
2026-09-01 08:30	24.2	66	24.033	1	0.9	0
2026-09-01 08:35	24.8	67	24.3	1	0.95	0
2026-09-01 08:40	25.2	68	24.733	1	1	0
2026-09-01 08:45	25.6	68	25.2	1	1.05	0
2026-09-01 08:50	25.9	69	25.567	1	1.1	0
2026-09-01 08:55	26.4	70	25.967	1	1.14	0

A base final mantém os dados tratados e os atributos criados. O arquivo dados_sala_inteligente_preparados.csv acompanha este material e contém também as colunas normalizadas e padronizadas.

17. Resolução comentada: o que foi descoberto

- A base bruta tinha 25 linhas, mas 1 era uma duplicata exata; restaram 24 registros únicos.
- A leitura 75,2 estava em Fahrenheit e corresponde a 24,0 °C; não era um evento térmico extremo.
- A leitura 240 °C foi considerada incompatível com o cenário e convertida em ausente antes da interpolação.
- A temperatura e a umidade ausentes foram preenchidas por interpolação linear porque as lacunas eram curtas e havia valores válidos próximos.
- A média móvel de três leituras reduziu pequenas oscilações sem apagar a série original.
- O indicador didático encontrou quatro intervalos de cinco minutos em que a sala estava vazia, mas a potência média permanecia em pelo menos 0,80 kW.
- A divisão treino/teste respeitou o tempo e o ajuste de escala foi calculado apenas com o treino, evitando vazamento de dados.
-

Interpretação responsável: A conclusão correta não é “houve desperdício comprovado”. O dado indica um padrão que merece investigação: climatização, computadores ou outro equipamento podem continuar ligados após a saída das pessoas.

18. Checklist de testes

Teste	Ação	Resultado esperado	Conceito
1	Contar linhas antes/depois	25 → 24	duplicidade
2	Contar ausentes iniciais	temp=1; umid=1	completude
3	Verificar unidade F	1 registro	consistência
4	Converter 75,2 °F	24,0 °C	transformação
5	Marcar 240 °C	NA	validade
6	Interpolar temp/umid	sem NA nessas colunas	imputação
7	Calcular temp_mm3	nova coluna	ruído

8	Criar desperdício	4 intervalos	feature engineering
9	Separar treino/teste	18 / 6	validação temporal
10	Ajustar scaler	fit somente no treino	evitar vazamento

19. Código completo da solução

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# 1) carregar
df = pd.read_csv('dados_sala_inteligente_brutos.csv')

# 2) tempo e ordenação
df['timestamp'] = pd.to_datetime(df['timestamp'])
df = df.sort_values('timestamp').reset_index(drop=True)

# 3) duplicidades
df = df.drop_duplicates().reset_index(drop=True)

# 4) unidade de temperatura
mask_f = df['unidade_temp'].eq('F')
df.loc[mask_f, 'temp'] = (df.loc[mask_f, 'temp'] - 32) * 5/9
df.loc[mask_f, 'unidade_temp'] = 'C'

# 5) valores impossíveis -> ausentes
df.loc[(df['temp'] < 0) | (df['temp'] > 50), 'temp'] = pd.NA

# 6) interpolação
df['temp'] = df['temp'].interpolate(method='linear')
df['umid'] = df['umid'].interpolate(method='linear')

# 7) suavização
df['temp_mm3'] = df['temp'].rolling(3, min_periods=1).mean()

# 8) atributos
df['hora'] = df['timestamp'].dt.hour
df['turno'] = 'manha'
df['energia_kwh_intervalo'] = df['consumo_kw'] * (5/60)
df['desperdicio'] = ((df['presenca'] == 0) & (df['consumo_kw'] >= 0.80)).astype(int)
df['temp_variacao'] = df['temp'].diff().fillna(0)

# 9) divisão temporal
colunas = ['temp_mm3', 'umid', 'consumo_kw']
split = int(len(df) * 0.75)
treino = df.iloc[:split].copy()
teste = df.iloc[split:].copy()

# 10) padronização sem vazamento
scaler = StandardScaler()
scaler.fit(treino[colunas])
treino[['c' + '_z' for c in colunas]] = scaler.transform(treino[colunas])
teste[['c' + '_z' for c in colunas]] = scaler.transform(teste[colunas])

# 11) normalização para comparação
minmax = MinMaxScaler()
minmax.fit(treino[colunas])
treino[['c' + '_minmax' for c in colunas]] = minmax.transform(treino[colunas])
teste[['c' + '_minmax' for c in colunas]] = minmax.transform(teste[colunas])

# 12) recompor e salvar
resultado = pd.concat([treino, teste]).sort_index()
resultado.to_csv('dados_sala_inteligente_preparados.csv', index=False)
```

```
print(resultado.head())  
print('Intervalos de possível desperdício:', resultado['desperdicio'].sum())
```

20. Desafio para os alunos

Depois de reproduzir a solução, faça uma cópia do notebook e implemente as seguintes alterações sem copiar diretamente o gabarito:

- Mude o limiar de possível desperdício para 0,75 kW e compare quantos intervalos passam a ser sinalizados.
- Crie uma coluna conforto_termico com três categorias: confortável, atenção e quente, justificando os limites escolhidos para o exercício.
- Calcule também uma mediana móvel de três leituras e compare com a média móvel em presença de um pico artificial.
- Crie um gráfico mostrando apenas o período em que presença = 0.
- Explique por que ajustar StandardScaler com toda a base antes de separar treino e teste seria metodologicamente incorreto.

21. Materiais de apoio e referências

Material-base da disciplina: apresentação “Pré-processamento de dados em IoT — limpeza, ruídos, valores ausentes, transformação e padronização”. O exercício segue especialmente os tópicos de pipeline de preparação, qualidade dos dados, tratamento de ausentes, ruído, transformação temporal, criação de atributos, normalização/padronização, vazamento de dados e prática guiada com Google Colab/Python.

Documentações úteis para continuação da prática: pandas (missing data, interpolate, rolling e resample), scikit-learn (StandardScaler e MinMaxScaler) e Orange Data Mining (Preprocess).

Sugestão ao professor: Uma boa estratégia de aula é executar o código em blocos. Antes de cada bloco, peça aos alunos que prevejam o efeito esperado; depois, comparem a previsão com o resultado.